

CyberSim — Official Documentation

Autonomous LLM-Driven Cyberattack Simulation Framework — Version 0.1.0

CyberSim — Official Documentation

Project: CyberSim — Autonomous LLM-Driven Cyberattack Simulation Framework Version: 0.1.0

Audience: Diploma reviewers, security researchers, lab operators Status: Reference implementation for an academic thesis

Table of Contents

- 1 Overview
- 2 Architecture
- 3 System Requirements
- 4 Installation
- 5 Configuration
- 6 Starting the Stack
- 7 Connecting the Client to the Server
- 8 Operating the Framework
- 9 Tool Registry Reference
- 10 ReAct Protocol Reference
- 11 REST and WebSocket API Reference
- 12 Sandbox and Authorization Model
- 13 Report Generation
- 14 Troubleshooting
- 15 Security and Ethics
- 16 Frequently Asked Questions
- 17 Glossary

1. Overview

CyberSim is a high-performance research framework that places a local Granite large language model in the role of an autonomous offensive-security operator. The model reasons under a strict ReAct (Reason → Act → Observe) discipline, chooses a tool from a curated registry, watches the streamed stdout/stderr of the spawned subprocess, and adapts its next step accordingly. Every action is gated by a sandbox guard that enforces an operator-supplied target allow-list.

A FastAPI / WebSocket backend hosts the agent and the tools. A PyQt6 desktop client (Windows-native, dark theme) provides a live operator console with a streaming terminal, a target manager with JSON/CSV import-export, a file upload dashboard, and a multi-format report builder (Markdown, PDF, JSON).

CyberSim is meant for educational use in fully isolated laboratory networks. The sandbox refuses to act on any target that is not explicitly authorized.

High-level Capabilities

- Autonomous ReAct loop with self-debugging across tool errors.
- Eight integrated pentest tools: Nmap, Gobuster / dirsearch, Nuclei, SQLmap,

Hydra, a pure-Python DOM analyser, Metasploit RPC, and post-exploitation enumeration.

- Real-time WebSocket telemetry from server to client.
- Multi-format reporting with CVSS aggregation and success-rate metrics.
- Strict allow-list-based sandbox.
- Modular layout — every tool is a single subclass of BaseTool.

2. Architecture



Module Layout

Path	Responsibility
server/	FastAPI app, session manager, REST + WebSocket router, reports
client/	PyQt6 desktop application
client/widgets/	Dashboard, terminal, targets panel, session panel, report panel
agent_logic/	OllamaManager (LLM gateway) and ReActAgent (reasoning loop)
tools/	Base classes, registry, sandbox guard, and 8 tool wrappers
config/	settings.py and system_prompt.md
reports/	Generated reports
uploads/	Operator-uploaded scripts, exploits, wordlists
data/	Sample target lists
docs/	This documentation in markdown / DOCX / PDF / TXT

3. System Requirements

Operating System

- Windows 11 Pro (primary supported platform).
- Windows 10 (works, untested in diploma demos).
- Linux distributions with Python 3.13 and Qt 6 (works, untested).

Software

Component	Version	Purpose
Python	3.13+	Runtime
Ollama	latest	Hosts the Granite LLM
Nmap	7.94+	Network reconnaissance
Gobuster	3.6+ (or dirsearch)	Directory brute forcing
Nuclei	3.2+	Vulnerability scanning
SQLmap	1.8+	SQL injection auditing
Hydra	9.5+	Protocol authentication brute force
Metasploit RPC	6.4+ (optional)	Exploit framework integration

Hardware

- 16 GB RAM recommended (Granite uses ~6-8 GB).
- 20 GB free disk space for the Granite model and intermediate data.
- A discrete GPU is optional but accelerates LLM inference dramatically.

Network

- An isolated lab subnet. CyberSim must never be pointed at the public

internet without operator-controlled scope verification.

4. Installation

4.1 Install Python 3.13

Install from <<https://www.python.org/downloads/>> with the "Add Python to PATH" option enabled. Verify:

```
py -3.13 --version
```

4.2 Install Ollama

Download the Windows installer from <<https://ollama.com/download>>. After install, start the daemon:

```
ollama serve
```

(In normal Windows installs the daemon starts automatically on login.)

Pull the Granite model:

```
ollama pull granite3.1-dense:latest
```

Verify:

```
ollama list
```

4.3 Install Pentest CLIs

The following must be discoverable on PATH for the corresponding tool to be usable. CyberSim degrades gracefully (returns NOT_INSTALLED) if any binary is missing, so you can install only the subset you need.

Tool	Windows install hint
nmap	< https://nmap.org/download.html >
gobuster	< https://github.com/OJ/gobuster/releases >
dirsearch	pip install dirsearch (used if gobuster missing)
nuclei	< https://github.com/projectdiscovery/nuclei/releases >
sqlmap	pip install sqlmap or clone the repository
hydra	Ship from a Kali WSL distro or build with cygwin
metasploit	< https://www.metasploit.com/download >

If you run CyberSim from a WSL or Kali side-car, you can expose the binaries through PATH.

4.4 Clone the Project and Install Python Dependencies

```
cd G:\Diploma
py -3.13 -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install --upgrade pip
pip install -r requirements.txt
```

Optional extras:

```
pip install pymetasploit3 # enables msf_exploit tool
pip install paramiko      # enables remote SSH post-exploit mode
pip install python-docx   # used by the documentation generator
```

4.5 Confirm the Installation

```
python -c "import server, client, tools, agent_logic; print('imports OK')"
```

5. Configuration

CyberSim reads its configuration from environment variables. Defaults are suitable for a single-host lab. Copy .env.example to .env and edit it, or export the variables in your shell.

5.1 Ollama Settings

Variable	Default	Description
OLLAMA_HOST	http://127.0.0.1:11434	Ollama HTTP endpoint
OLLAMA_MODEL	granite3.1-dense:latest	Primary Granite tag
OLLAMA_FALLBACK_MODEL	granite3.1-dense:8b	Fallback if primary tag is missing locally
OLLAMA_TEMPERATURE	0.2	Sampling temperature

Variable	Default	Description
OLLAMA_TOP_P	0.9	Nucleus sampling parameter
OLLAMA_NUM_CTX	8192	Context window in tokens
OLLAMA_TIMEOUT	180	Request timeout in seconds
OLLAMA_KEEP_ALIVE	30m	How long Ollama keeps the model resident

5.2 Server Settings

Variable	Default	Description
CYBERSIM_HOST	127.0.0.1	FastAPI bind address
CYBERSIM_PORT	8765	FastAPI bind port
CYBERSIM_RELOAD	false	Set true to enable Uvicorn auto-reload

5.3 Agent Settings

Variable	Default	Description
AGENT_MAX_ITERATIONS	25	Hard cap on ReAct iterations per session
AGENT_REACT_PAUSE	0.0	Optional sleep between iterations (debug aid)

5.4 Metasploit RPC Settings (optional)

Variable	Default	Description
MSFRPC_HOST	127.0.0.1	msfrpcd host
MSFRPC_PORT	55553	msfrpcd TCP port
MSFRPC_USER	msf	msfrpcd username
MSFRPC_PASSWORD	_(unset)_	When unset the msf tool reports NOT_INSTALLED
MSFRPC_SSL	true	Use TLS for msfrpcd

Start msfrpcd in another terminal:

```
msfrpcd -P <password> -S -a 127.0.0.1
```

5.5 Tweaking the System Prompt

The agent's identity, mission, and safety rails are defined in config/system_prompt.md. Edit the file and restart the server to take effect — no code change required.

6. Starting the Stack

6.1 Start Ollama

```
ollama serve          # if not already running as a service
ollama list           # confirm granite3.1-dense:latest is present
```

6.2 Start the Backend Server

```
.\run_server.bat
```

You should see:

```
INFO CyberSim server ready on http://127.0.0.1:11434 using model=granite3.1-dense:latest
INFO Uvicorn running on http://127.0.0.1:8765
```

A quick health check:

```
curl http://127.0.0.1:8765/health
```

6.3 Start the PyQt6 Client

In a second terminal:

```
.\run_client.bat
```

The CyberSim window opens. If the status bar shows "server: OK" you are ready.

7. Connecting the Client to the Server

By default the client looks for the server on 127.0.0.1:8765. To target a remote server, export environment variables before launching the client:

```
$env:CYBERSIM_HOST = "10.10.10.20"
$env:CYBERSIM_PORT = "8765"
.\run_client.bat
```

The client uses HTTP for control plane traffic (start session, list reports, upload files, etc.) and a WebSocket for the live event stream:

```
HTTP : http://10.10.10.20:8765/api/...
WS    : ws://10.10.10.20:8765/ws/sessions/<session-id>
```

Connectivity Troubleshooting

- 1 Visit `http://<host>:8765/health` in a browser — you should see JSON.
- 2 Open Windows Defender Firewall and allow inbound TCP 8765.
- 3 If the client reports "server unreachable", check the server log for a

crash; the most common cause is Ollama being unreachable.

- 1 Use `netstat -an | findstr 8765` to confirm the server is listening.

8. Operating the Framework

The CyberSim client groups operator activity into three areas: Dashboard, Live Terminal, and Sessions / Report.

8.1 Compose the Objective

In the Dashboard, fill in the Attack Objective text field with a clear description of what you want the autonomous agent to accomplish. Examples:

> "Map the web stack on http://lab.local, enumerate paths, audit any > parameters you find for SQL injection, and produce a remediation report."

> "Discover services on 10.10.10.0/24, focus on hosts with port 445 open, > identify the OS version, and check for SMB-related vulnerabilities."

8.2 Authorize Targets

Add targets to the Authorized Targets panel:

- Single IP — ``10.10.10.5``
- CIDR network — ``10.10.10.0/24``
- URL — ``http://lab.local/admin``
- Hostname — ``vulnbox.lab``

You can import a list from JSON or CSV (see `data/targets_sample.json` and `data/targets_sample.csv` for the expected layout).

If the allow-list is empty, every tool call is refused.

8.3 Upload Custom Payloads (optional)

In the Dashboard's Scripts / Exploits / Wordlists panel you can upload custom wordlists, scripts, or exploit payloads. The agent can reference these in tool arguments (for example: `wordlist=./uploads/custom.txt`).

8.4 Launch the Session

Press Launch ReAct Session. The client switches to the Live Terminal, opens a WebSocket to the server, and streams every event:

- ``session_start`` — operator objective + scope.
- ``llm_thought`` — the model's reasoning for this turn.
- ``tool_call`` — chosen tool and validated arguments.
- ``tool_stream`` — line-buffered stdout/stderr from the subprocess.
- ``tool_result`` — structured summary returned to the agent.
- ``metrics`` — iterations, tool calls, latency, sandbox blocks.
- ``final_answer`` — the agent's final report (markdown).
- ``session_end`` — terminal event.

8.5 Inspect the Session

Switch to the Sessions tab to see previous attacks, their state, and the number of iterations. Select a session and click Open Terminal to replay its event log, or Build Report to render and export a report.

8.6 Export a Report

In the Report tab choose a format (Markdown, PDF, or JSON), press Build to compile, then Download to save the artefact.

9. Tool Registry Reference

All tools share a common JSON-schema-style parameter contract. The LLM sees the full catalog at the start of every session.

9.1 ``nmap_scan`` — Network Reconnaissance

Parameter	Type	Required	Description
target	string	yes	Host, hostname, or CIDR (must be in scope)
ports	string	no	Port spec like 1-1024 or 22,80,443
profile	string	no	quick, service, aggressive, discovery, vuln
extra_flags	string	no	Extra nmap flags (restricted to a safe subset)
timeout_s	number	no	Hard timeout, default 600 s

Returns structured host/port/service data.

9.2 `dir\_brute` — Directory Enumeration

Uses Gobuster, falling back to dirsearch.

Parameter	Type	Required	Description
target	string	yes	Full base URL
wordlist	string	no	Path to a wordlist
extensions	string	no	php,asp,txt style list
threads	integer	no	Concurrency, default 30
status_codes	string	no	Default 200,204,301,302, 307,401,403
timeout_s	number	no	Hard timeout, default 600 s

9.3 `vuln\_scan` — Nuclei Template Scan

Parameter	Type	Required	Description
target	string	yes	Full URL
severity	string	no	Comma list, default medium,high,critical
tags	string	no	Nuclei tag filter
rate_limit	integer	no	Requests per second, default 150
timeout_s	number	no	Hard timeout, default 900 s

9.4 `sqlmap\_audit` — SQL Injection

Parameter	Type	Required	Description
target	string	yes	URL with parameters
data	string	no	POST body
level	integer	no	1-5, default 2
risk	integer	no	1-3, default 1
technique	string	no	Subset of BEUSTQ, default BEU
dump	bool	no	Dump discovered tables if true
timeout_s	number	no	Hard timeout, default 900 s

9.5 `auth\_bruteforce` — Hydra

Parameter	Type	Required	Description
target	string	yes	Host or IP
service	string	yes	ssh, ftp, http-post-form, smb, etc.
user	string	no	Single username
user_list	string	no	Path to a username list
pass_list	string	no	Path to a password list
port	integer	no	Service port
module_opts	string	no	Extra service-module options
tasks	integer	no	Parallel tasks, default 8
timeout_s	number	no	Hard timeout, default 600 s

9.6 `web\_analyze` — DOM and Header Analyser

Pure-Python tool — no external CLI required.

Parameter	Type	Required	Description
target	string	yes	Full URL
follow_redirects	bool	no	Default true
max_links	integer	no	Default 200
user_agent	string	no	Custom User-Agent
timeout_s	number	no	Default 30 s

9.7 `msf\_exploit` — Metasploit RPC

Requires msfrpcd running and MSFRPC_PASSWORD exported.

Parameter	Type	Required	Description
target	string	yes	RHOST
module	string	yes	Full module path
module_type	string	no	exploit, auxiliary, or post
options	object	no	Datastore overrides
payload	string	no	Payload override
wait_seconds	number	no	Time to wait for sessions, default 25 s

9.8 `post\_exploit\_enum` — Post-Exploitation

Parameter	Type	Required	Description
mode	string	yes	local or remote
target	string	conditional	Required when mode is remote
username	string	conditional	SSH user for remote mode
password	string	conditional	SSH password (or use key_path)
key_path	string	conditional	Path to SSH private key
timeout_s	number	no	Per-command timeout

10. ReAct Protocol Reference

Every LLM turn must end with exactly one JSON object on the final line:

```
{
  "thought": "Nmap revealed port 80 – enumerate web paths next.",
  "action": "tool",
  "tool": "dir_brute",
  "arguments": {"target": "http://10.10.10.5/", "threads": 30}
}
```

To finish:

```
{
  "thought": "All checks complete.",
  "action": "finish",
  "final_answer": "## Findings\n- High severity: ..."
}
```

The framework feeds the next observation back to the model as a user turn:

```
Observation:
[nmap_scan :: ok] (2.3s)
nmap service scan of 10.10.10.5: 1 host(s), 3 open port(s) ...
```

If the model emits malformed JSON, the framework:

- 1 Salvages the last balanced `{...}` substring.
- 2 If still invalid, asks the model to retry with strict JSON.

3 Aborts after `json\_retry\_limit` failures.

11. REST and WebSocket API Reference

Base URL: `http://<host>:<port>`.

Method	Path	Description		
GET	/health	Server and Ollama health		
GET	/api/tools	LLM-facing tool catalog		
POST	/api/sessions	Start a new attack session		
GET	/api/sessions	List sessions		
GET	/api/sessions/{id}	Full session payload		
POST	/api/sessions/{id}/stop	Cancel a running session		
POST	/api/sessions/{id}/report?fmt=markdown	pdf	json	Build report
GET	/api/sessions/{id}/report/download?fmt=...	Download report		
POST	/api/uploads	Upload a file		
GET	/api/uploads	List uploads		
POST	/api/llm/chat	Direct chat with the LLM		
WS	/ws/sessions/{id}	Stream session events		

POST /api/sessions (request)

```
{
  "objective": "Map and audit http://lab.local",
  "targets": ["lab.local", "10.10.10.5"]
}
```

POST /api/sessions (response)

```
{ "session_id": "ab12cd34ef56", "state": "running" }
```

WebSocket /ws/sessions/{id}

Each message is a JSON-encoded AgentEvent:

```
{
  "type": "tool_result",
  "payload": {
    "iteration": 3,
    "tool": "nmap_scan",
  }
}
```

```

    "status": "ok",
    "summary": "1 host, 3 open ports",
    "duration_s": 2.3
  },
  "ts": "2025-05-15T17:00:00.123456+00:00"
}

```

12. Sandbox and Authorization Model

12.1 Allow-list

SandboxGuard.set_targets rebuilds the allow-list each session:

- Single IPs (`10.10.10.5`)
- CIDR ranges (`10.10.10.0/24`)
- Hostnames (`vulnbox.lab`)
- URLs — the host portion is extracted (`http://lab.local/admin/` becomes

lab.local).

12.2 Enforcement

Each tool declares which argument fields contain targets via ToolSchema.target_fields. The registry resolves the operator-supplied value to a host string, attempts DNS resolution, and asserts that the result is in the allow-list.

- Empty allow-list — every call is refused.
- Out-of-scope target — the registry returns `ToolStatus.BLOCKED` and emits a

sandbox_block event so the LLM can adapt.

12.3 Forbidden Behaviours

The system prompt explicitly forbids:

- Acting on hosts outside the allow-list.
- Exfiltrating data outside the lab.
- Destructive payloads (wipers, ransomware, persistence rootkits).
- Denial-of-service flooding.

13. Report Generation

The report generator (server.report_generator.ReportGenerator) produces:

- **Markdown** — for documentation, GitHub, or quick reading.
- **PDF** — operator-grade, via ReportLab. Includes tables for metrics and

findings.

- **JSON** — full machine-readable session payload.

13.1 Metrics Aggregated

- Tool call totals: total, success, errors, blocked.
- Success rate (success / total).
- Number of findings (with severity).
- Maximum and average CVSS scores.

13.2 Findings Sources

- `nuclei` findings (template id, severity, CVSS, CVE).
- SQLmap-confirmed injections.
- Hydra-confirmed credentials.
- Metasploit-opened sessions (treated as critical-severity findings).

14. Troubleshooting

Symptom	Likely cause and fix
"server unreachable" in the client	The backend is not running, or it is bound to a different host/port. Check run_server.bat logs.
Server log: "Ollama chat failed: ConnectError"	Ollama daemon is not running, or OLLAMA_HOST is wrong.
Tool returns NOT_INSTALLED	The corresponding CLI is missing from PATH. Install it or skip that tool.
Tool returns BLOCKED	Target not in the allow-list. Add it to the Targets panel.
Hydra reports "no password list provided"	Install SecLists or supply pass_list explicitly.
msf_exploit returns NOT_INSTALLED	msfrpcd is not running, or MSFRPC_PASSWORD is unset, or pymetasploit3 is missing.
Client crashes with "No module named PyQt6"	Activate .venv and run pip install -r requirements.txt.
ReAct loop never finishes	Increase AGENT_MAX_ITERATIONS, or refine the objective so the model has clearer success conditions.

15. Security and Ethics

CyberSim is a research and education tool. Improper use can violate computer-crime laws in nearly every jurisdiction. The following responsibilities fall on the operator:

- Maintain an isolated lab network.
- Maintain written authorization for every target you load.
- Never disable the sandbox guard in production environments.
- Treat the generated reports as confidential — they may contain

exploitation evidence.

16. Frequently Asked Questions

Q. Can I use a different LLM? A. Yes. Set OLLAMA_MODEL to any other Ollama-hosted tool-capable model. The ReAct protocol is model-agnostic; we recommend Granite for the diploma demo.

Q. Can I add a new tool? A. Yes. Create a subclass of BaseTool, declare a ToolSchema, implement async def run(...), then register it in tools/__init__.py::build_default_registry.

Q. Does CyberSim work offline? A. Yes — Ollama and every pentest tool run locally. The only external traffic is what the pentest tools themselves generate against the targets you authorize.

Q. How can I tune the agent's behaviour? A. Edit config/system_prompt.md (mission, tone, safety rails) and the environment variables in section 5 (temperature, context window, iteration limit).

Q. Can I export the live terminal log? A. Yes. Either copy it from the client window, or request the JSON report — it includes every emitted event.

17. Glossary

- **ReAct loop** — Reason → Act → Observe; a structured prompting pattern

where the model alternates between chain-of-thought reasoning and tool invocations.

- **Tool registry** — the table of executable capabilities exposed to the

LLM, with JSON schema and sandbox metadata.

- **Sandbox guard** — the component that enforces the operator-supplied

target allow-list before any tool runs.

- **CVSS** — Common Vulnerability Scoring System; standardised severity

metric used in the report aggregation step.

- **msfrpcd** — Metasploit's RPC daemon, enabling remote control of the

framework via MessagePack RPC.

- **WebSocket telemetry** — the low-latency channel from the FastAPI backend

to the PyQt6 client.

End of documentation.